

# Topic 4: Data Cleaning and Preparation

Missing values and duplicates

Igor Razgon

*igor.razgon@durham.ac.uk*

## Content

1. Taking care of missing values
2. Removal of duplicates

# Taking care of missing values

# Where do missing values come from?

- Suppose that we initialize a *DataFrame* instance by reading from a .csv file where some cells are blank.
- These values will appear in the data frame as *missing*.
- If you print the instance, the respective values will appear as *NaN* standing for 'Not a Number'.
- **Remark:** the above is not the *only* possible way for occurrence missing values. They can also occur as a result of the outer merge operation.

# Example: a *DataFrame* instance with missing values

```
pdmv = pd.read_csv('missvalues.csv')
```

```
pdmv
```

|   | col1 | col2 | col3 |
|---|------|------|------|
| 0 | 1    | 2.0  | 3    |
| 1 | 4    | NaN  | 6    |

# Taking care of missing values

- Taking care of missing values is an important part of data cleaning process.
- Possible approaches include:
  - Doing nothing (ignoring the presence of missing values)
  - Dropping some of components of a data frame that contain missing values.
  - Filling missing values.
- The choice of the approach depends on the application.
- We will consider several applications, outline the corresponding reasonable approach, and demonstrate their Python implementations.

# Ignoring missing values may be reasonable

- Consider a data frame is a 24 months salary survey for a big organization (say 1000 employees).
- The Employees correspond to the rows, and the months correspond to the columns.
- For many employees (say up to 800) salary records are sporadically missing (say up to 3 months per employee).
- Suppose we need to compute average salaries of the employees.
- Then it is reasonable to compute the averages ignoring the missing entries.

# Example: averages ignoring the missing values

```
pdsalary = pd.read_csv('missingsalary.csv')
```

```
#Computing means per row
```

```
#By default, the function mean ignores missing values
```

```
means=pdsalary[pdsalary.columns[1:]].mean(1)
```

```
pdsalary
```

|   | id | salary June | salary July | salary August |
|---|----|-------------|-------------|---------------|
| 0 | 23 | NaN         | 10000.0     | 12000.0       |
| 1 | 47 | 8000.0      | NaN         | 6000.0        |
| 2 | 96 | 5000.0      | 7000.0      | NaN           |

```
means
```

|   |         |
|---|---------|
| 0 | 11000.0 |
| 1 | 7000.0  |
| 2 | 6000.0  |

```
dtype: float64
```



# Dropping rows or columns of a DataFrame

- Suppose that for the same salary survey as in the previous example, one of the following happens.
  - Only a few employees (say 100 out of 1000 have missing entries, but the proportion of missing entries for each these employees is large (say, more than 50%).
  - Only a few months (say 3 out of 24) are missing, but the proportion of missing entries is large.
- In this case, it is reasonable to remove the respective rows and columns with the missing values.

# Example: dropping rows or columns with missing values

```
pdmissing = pd.read_csv('missvalues.csv')
```

*pdmissing*

|   | col1 | col2 | col3 |
|---|------|------|------|
| 0 | 1    | 2.0  | 3    |
| 1 | 4    | NaN  | 6    |

*#Dropping rows containing missing values*

```
pdfullrows=pdmissing.dropna()
```

*pdfullrows*

|   | col1 | col2 | col3 |
|---|------|------|------|
| 0 | 1    | 2.0  | 3    |

*#Dropping columns containing Null values*

*#Note axis=1 as the parameter of dropnae function*

```
pdfullcolumns=pdmissing.dropna(axis=1)
```

*pdfullcolumns*

|   | col1 | col3 |
|---|------|------|
| 0 | 1    | 3    |
| 1 | 4    | 6    |

# Reducing only rows with all values missing

- Let's consider again the scenario with computing averages.
- Suppose that a small number of employees are not surveyed: in the corresponding rows all the values are missing.
- Then it is reasonable to remove these rows before any further analysis.

# Example: removal of rows with all values missing

```
pdmissing = pd.read_csv('missingrow.csv')
```

```
pdnoallmissing=pdmissing.dropna(how='all')
```

*pdmissing*

|   | col1 | col2 | col3 |
|---|------|------|------|
| 0 | 1.0  | 2.0  | 3.0  |
| 1 | NaN  | NaN  | NaN  |
| 2 | 4.0  | NaN  | 6.0  |

*pdnoallmissing*

|   | col1 | col2 | col3 |
|---|------|------|------|
| 0 | 1.0  | 2.0  | 3.0  |
| 2 | 4.0  | NaN  | 6.0  |

## *Filling all missing entries with the same value*

- Consider a DataFrame with:
  - rows corresponding to students
  - columns corresponding to modules.
- Some values are missing meaning that the corresponding student did not take the corresponding module.
- If all the values need to be filled (e.g. for the graduation exam board) then it is reasonable to fill the missing values with zeroes.

# Example: filling all the missing values with zeroes

```
pdmarks = pd.read_csv('marksmissing.csv'):
```

```
pdmarks
```

|   | Name     | Gender | DOB        | Maths | Physics | Chemistry | English | Biology |
|---|----------|--------|------------|-------|---------|-----------|---------|---------|
| 0 | John     | M      | 05/04/1988 | 55    | 45.0    | 56        | 87.0    | 21      |
| 1 | Suresh   | M      | 04/05/1987 | 75    | 96.0    | 78        | NaN     | 90      |
| 2 | Jessica  | F      | 12/08/1990 | 78    | 96.0    | 86        | 63.0    | 54      |
| 3 | Jennifer | F      | 02/09/1989 | 58    | NaN     | 78        | 46.0    | 96      |
| 4 | Annu     | F      | 05/04/1988 | 45    | 87.0    | 52        | 89.0    | 55      |
| 5 | Ritesh   | M      | 25/05/1989 | 54    | NaN     | 87        | 56.0    | 25      |
| 6 | Farha    | F      | 12/08/1990 | 55    | 63.0    | 89        | 75.0    | 78      |
| 7 | Mukesh   | M      | 02/09/1989 | 96    | 46.0    | 77        | NaN     | 58      |

```
#Filling missing marks with zeroes
```

```
pdmarksfull=pdmarks.fillna(0)
```

```
pdmarksfull
```

|   | Name     | Gender | DOB        | Maths | Physics | Chemistry | English | Biology |
|---|----------|--------|------------|-------|---------|-----------|---------|---------|
| 0 | John     | M      | 05/04/1988 | 55    | 45.0    | 56        | 87.0    | 21      |
| 1 | Suresh   | M      | 04/05/1987 | 75    | 96.0    | 78        | 0.0     | 90      |
| 2 | Jessica  | F      | 12/08/1990 | 78    | 96.0    | 86        | 63.0    | 54      |
| 3 | Jennifer | F      | 02/09/1989 | 58    | 0.0     | 78        | 46.0    | 96      |
| 4 | Annu     | F      | 05/04/1988 | 45    | 87.0    | 52        | 89.0    | 55      |
| 5 | Ritesh   | M      | 25/05/1989 | 54    | 0.0     | 87        | 56.0    | 25      |
| 6 | Farha    | F      | 12/08/1990 | 55    | 63.0    | 89        | 75.0    | 78      |
| 7 | Mukesh   | M      | 02/09/1989 | 96    | 46.0    | 77        | 0.0     | 58      |

# Averages per row as missing values

- Let us get back to the first scenario with salaries (small proportion of entries per employee are missing).
- Assume now that we cannot ignore the missing values (e.g. due to some procedural reason).
- Then it is reasonable to fill missing entries of each employee with the **average** salary of that employee.

# Example: filling all the missing values with averages per row

```
pdsalary = pd.read_csv('missingsalary.csv')
```

```
#Salaries sheet with missing values
```

```
pdsalary
```

|   | id | salary June | salary July | salary August |
|---|----|-------------|-------------|---------------|
| 0 | 23 | NaN         | 10000.0     | 12000.0       |
| 1 | 47 | 8000.0      | NaN         | 6000.0        |
| 2 | 96 | 5000.0      | 7000.0      | NaN           |

```
#Repalcing missing values with means
```

```
for i in pdsalary.index:
```

```
    pdsalary.loc[i]=pdsalary.loc[i].fillna(means[i])
```

```
pdsalary
```

|   | id | salary June | salary July | salary August |
|---|----|-------------|-------------|---------------|
| 0 | 23 | 11000.0     | 10000.0     | 12000.0       |
| 1 | 47 | 8000.0      | 7000.0      | 6000.0        |
| 2 | 96 | 5000.0      | 7000.0      | 6000.0        |



# Filling missing entries by 'fill forward' method

- Suppose that a data frame contains temperature observations taken once in 30 minutes at the same location.
- However, there are small missing gaps in the observation.
- It is then reasonable to fill each gap with the last temperature preceding the gap.

# Example of the 'fill forward method

```
pdtemp = pd.read_csv('misstemp.csv')
```

```
pdtemp
```

*#Filling columns forward without propagation limits*

```
fwtemp=pdtemp.fillna(method='ffill')
```

```
fwtemp
```

*#Filling forward with propagation*

```
fwlimit=pdtemp.fillna(method='ffill',limit=1)
```

```
fwlimit
```

|   | time  | temperature |
|---|-------|-------------|
| 0 | 10:00 | 1.0         |
| 1 | 10:30 | 2.0         |
| 2 | 11:00 | 2.0         |
| 3 | 11:30 | NaN         |
| 4 | 12:00 | 3.0         |
| 5 | 12:30 | 4.0         |
| 6 | 13:00 | 4.0         |
| 7 | 13:30 | NaN         |

|   | time  | temperature |
|---|-------|-------------|
| 0 | 10:00 | 1.0         |
| 1 | 10:30 | 2.0         |
| 2 | 11:00 | 2.0         |
| 3 | 11:30 | 2.0         |
| 4 | 12:00 | 3.0         |
| 5 | 12:30 | 4.0         |
| 6 | 13:00 | 4.0         |
| 7 | 13:30 | 4.0         |

|   | time  | temperature |
|---|-------|-------------|
| 0 | 10:00 | 1.0         |
| 1 | 10:30 | 2.0         |
| 2 | 11:00 | NaN         |
| 3 | 11:30 | NaN         |
| 4 | 12:00 | 3.0         |
| 5 | 12:30 | 4.0         |
| 6 | 13:00 | NaN         |
| 7 | 13:30 | NaN         |

# *fillna()* and *dropna()* functions: general view

- *fillna()* and *dropna()* are functions with many parameters that allow adaptation to many circumstances related to taking care of missing values
- Please consult  
<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html>  
and <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html>  
for further information.

# Filling missing values

- Filling missing values is an important statistical task called **imputation**.
- There are deep methodologies for carrying out the imputation that outside the scope of this module.
- For further information, please consult and the corresponding Wikipedia page [https://en.wikipedia.org/wiki/Imputation\\_\(statistics\)](https://en.wikipedia.org/wiki/Imputation_(statistics)) that also provides links for further reading.

## *isnull()* function

- *isnull()* function can be applied to a specific row or column of a DataFrame.
- It returns a Boolean pattern, where:
  - *True* entries correspond to existing values
  - *False* entries correspond to missing values.
- Convenient to use for checking whether a specific cell of a DataFrame is missing.

# Example of *isnull()* function use

```
pdmv = pd.read_csv('missvalues.csv')
```

```
pdmv
```

|   | col1 | col2 | col3 |
|---|------|------|------|
| 0 | 1    | 2.0  | 3    |
| 1 | 4    | NaN  | 6    |

*#Column pattern for missing values*

```
misscolumn=pdmv['col2'].isnull()
```

```
nisscolumn
```

|   |       |
|---|-------|
| 0 | False |
| 1 | True  |

Name: col2, dtype: bool

*#Row pattern for missing values*

```
missrow=pdmv.loc[1].isnull()
```

```
nissrow
```

|      |       |
|------|-------|
| col1 | False |
| col2 | True  |
| col3 | False |

Name: 1, dtype: bool

# Removal of duplicates

# Duplicates

- Duplicates of a DataFrame are identical rows.
- Duplicates may also be *restricted* or *subject to a subset of columns*. In this case, the rows are identical in the specified set of columns but may be distinct otherwise.



# Example: demonstration of duplicates

*#Data Frame for duplicates testing*

```
pddupval = pd.read_csv('DuplDemoRow.csv')
```

```
pddupval
```

|   | col1 | col2 |
|---|------|------|
| 0 | 1    | 8    |
| 1 | 2    | 3    |
| 2 | 1    | 4    |
| 3 | 2    | 7    |
| 4 | 1    | 4    |

*#Boolean mask for full duplicates*

```
pddupval.duplicated()
```

```
0    False
1    False
2    False
3    False
4     True
dtype: bool
```

*#Boolean mask for column 1 duplicates only*

```
pddupval.duplicated(subset=['col1'])
```

```
0    False
1    False
2     True
3     True
4     True
dtype: bool
```

# The need to drop values

- Depending on the application, duplicates may need to be removed.
  - **Removal is needed:** the DataFrame provides the list of employees of some company. If some employee appears more than once, duplicate records need to be removed.
  - **Removal is not needed:** each row of a data frame corresponds to a shift and contains contact details of the employee who is in charge of the shift. There will be a lot of duplicate rows, and they are important part of the information.
- We will assume that the **removal is needed** and consider several approaches to its implementation.

# Example: removal of full duplicates

We continue on the previous example where the last row with entries 1 and 4 was duplicated.

*#Full duplicates dropped*

*dropped=pddupval.drop\_duplicates()*

*dropped*

|          | col1 | col2 |
|----------|------|------|
| <b>0</b> | 1    | 8    |
| <b>1</b> | 2    | 3    |
| <b>2</b> | 1    | 4    |
| <b>3</b> | 2    | 7    |

# Removal of duplicates subject to a subset of columns

- In *DuplDemoRow.csv*, consider removal of duplicates subject to *col1*.
- The values of *col2* may be distinct.
- For example, rows (1, 8) and (1, 4) are considered identical subject to column 1.
- After the duplicates removal, only one row with 1 in *col1* remains.
- What value in *col2* will this row have?

# Restricted functionality of *drop\_duplicates*

- *drop\_duplicates* has parameter *keep* that determines which value of col/2 to keep.
- *keep='first'* keeps that first occurring value (the default option)
- *keep='last'* keeps the last occurring value.

# Example: taking care of the remaining column

*#Data Frame for duplicates testing*

```
pddupval = pd.read_csv('DuplDemoRow.csv')
```

```
pddupval
```

|   | col1 | col2 |
|---|------|------|
| 0 | 1    | 8    |
| 1 | 2    | 3    |
| 2 | 1    | 4    |
| 3 | 2    | 7    |
| 4 | 1    | 4    |

*#col1 duplicates dropped. The first col2 entry left*

```
dropped1=pddupval.drop_duplicates(subset=['col1'])
```

```
dropped1
```

|   | col1 | col2 |
|---|------|------|
| 0 | 1    | 8    |
| 1 | 2    | 3    |

*#col1 duplicates dropped, the last col2 entry left*

```
dropped2=pddupval.drop_duplicates(subset=['col1'], keep='last')
```

```
Dropped2
```

|   | col1 | col2 |
|---|------|------|
| 3 | 2    | 7    |
| 4 | 1    | 4    |

## More complicated arrangements

- What if we want a more complicated function of the values of *col2* clustered by the same value of *col1*.
- For example, the first column may correspond to student IDs, and the second column may correspond to their exam marks for the same module.
- If they take several attempts, the **maximum** or **average** mark may need to be awarded.
- This can be addressed by the **groupby** function (to be considered next week).